

store

A simple CRUD REST API — Node.js, TypeScript, PostgreSQL

Reviewed 15 September 2026 · branch main @ 06ccfb5 · 40 TypeScript files, ~1,100 LOC

1. Scope and method

This review covers the whole repository at commit 06ccfb5 on main: the hand-rolled HTTP server and file-system route registry, the authentication and authorisation layer, the Zod validation schemas, the Postgres repositories, the SQL migrations and runner, and the test and tooling setup. Every source file was read in full; no third-party dependency audit was performed.

Verification performed during the review:

- `tsc --noEmit` — passes with no diagnostics.
- `vitest run --coverage` — 4 files, 35 tests, all passing in about 3 seconds.
- The postgres.js SET-clause builder was inspected directly in `node_modules` to confirm the behaviour described in M1.
- No live database was available, so runtime SQL behaviour is reasoned from the driver source and the migration DDL rather than executed.

Findings are rated by the damage they can do in production, not by how hard they are to fix. There are no findings rated Critical: nothing here hands an attacker the database or another user's account outright. The High band is where real money or real accounts are at risk.

2. Executive summary

This is a well-organised codebase for its size. The layering is deliberate — routes stay thin, repositories own SQL, Zod schemas sit at both the request and the row boundary, and authorisation is declared per method on each route rather than re-implemented in every handler. Parameterised queries are used consistently, so the obvious SQL-injection surface is closed. Login deliberately verifies against a dummy hash when the user is not found, which is a thoughtful touch most projects of this size miss. The custom router, registry and migration runner are all clean, readable code.

The gaps are concentrated in three places. First, the write paths have no transactions, so the cart's check-stock-then-update sequences can be raced into overselling. Second, the boundary between internal errors and HTTP responses leaks: raw Postgres error objects are serialised to clients, and several not-found cases surface as 400 validation errors instead. Third, there is almost no coverage where the risk actually lives — the 98% coverage figure is measured only over the four modules the tests import, and no route, repository or dispatch path is exercised at all.

AREA	ASSESSMENT	RATING
Architecture and layering	Clear separation of routes, repositories, validation and auth. Some leaky abstractions and a barrel-import cycle.	Strong
REST / API design	Non-RESTful paths, no pagination, inconsistent response envelopes, no PUT, no OPTIONS.	Needs work
Input validation	Zod at every entry point. Gaps: no password policy, price validated only as a string, mass assignment on <code>is_active</code> .	Good

AREA	ASSESSMENT	RATING
Data integrity	Good foreign-key and CHECK constraints. No transactions anywhere in the API; nullable columns the schemas treat as non-null.	Needs work
Security	Parameterised SQL, bcrypt, dummy-hash timing defence. Missing password policy, rate limiting, startup secret checks; error detail leaks.	Needs work
Error handling	Centralised and consistent in shape, but leaks driver internals and misclassifies status codes.	Mixed
Testing	35 well-written unit tests, but only over utils, parser and middleware. No route, repository or integration tests. No CI.	Weak
Tooling and docs	Prettier and strict TS flags in place. No README, no .env.example, no linter, no CI, no API docs.	Weak

3. Findings

Nineteen findings are detailed below, ordered by severity. The remaining lower-severity observations are collected in section 8 so this section stays readable.

High severity

HIGH

H1 — Raw Postgres error objects are serialised into HTTP responses

Where `src/lib/structures/Server.ts:83`

Issue Any `PostgresError` that escapes a handler is JSON-stringified straight into the response body and returned as 400.

SERVER.TS

```
} else if (err instanceof postgres.PostgresError) {
  this.logger.warn(`Postgres Error invoked for ${method}:${path}: ${err.message}`);
  return res.writeHead(400).end(JSON.stringify(err));
}
```

Impact Clients receive the driver's error object — message, constraint name, table, column, character position and the failing statement fragment. That is a free schema map for anyone probing the API, and it is reachable unauthenticated (a malformed register payload is enough). Separately, every database failure becomes a 400: a connection drop, a deadlock and a syntax error are all reported to the client as "your request was bad", which makes real outages invisible in client-side metrics.

Fix Never serialise a driver error. Map the small set of codes you actually handle to domain responses — 23505 to 409, 23503 to 400 or 404, 23514 to 400 — and let everything else fall through to a generic 500 carrying a correlation id, with the full detail in the log only.

HIGH

H2 — No password policy — an empty password is accepted

Where `src/lib/validation/authSchema.ts:16` (`registerSchema.password`)

Issue `password` is declared as `z.string("password is required")` with no length constraint, and nothing else in the register path applies one.

AUTHSCHEMA.TS

```
password: z.string("password is required"),
```

Impact `POST /api/auth/register` happily creates an account whose password is "" or "a". Those accounts are trivially guessable, and because there is no rate limiting (H7) nothing slows an attacker down. `bcrypt`'s cost factor protects the stored hash, not a password that can be guessed on the first attempt.

Fix Require a minimum length on the register schema — `z.string().min(12).max(128)` is a reasonable default — and reject the handful of obvious values. Keep `loginSchema` permissive so existing accounts can still authenticate; deriving it via `registerSchema.omit({ name: true })` couples the two, so split them.

HIGH

H3 — Cart stock checks are not atomic — concurrent requests can oversell

Where `src/api/cart/index.ts:14-27`, `src/api/cart/[productId].ts:6-24`, `cartRepository.ts`

Issue Every cart write is a read-then-write sequence across separate round trips with no transaction and no row lock. Add-to-cart reads product stock, reads the current cart quantity, compares, then upserts.

CART/INDEX.TS (POST)

```
const exists = await new ProductRepository(db).exists(data.product_id);
const currItemQuantity = (await cartRepo.findItem(user.id, data.product_id)?.quantity ?? 0;
if (exists.stock < currItemQuantity + data.quantity)
  throw new CustomError(409, "Requested quantity exceeds available stock");
const addToCart = await cartRepo.upsertItem(user.id, data.product_id, data.quantity);
```

Impact Two interleaved requests both see the old quantity and both pass the stock test. Because `upsertItem` adds to the existing row (`quantity = cart_items.quantity + excluded.quantity`), the result is a cart holding more units than exist. The `cart_items` CHECK constraint only enforces `quantity > 0`, so the database will not catch it. The same window exists on PATCH, and nothing in the schema ties cart quantity to product stock at all.

Fix Wrap each cart mutation in `db.begin()` and take the product row with `SELECT ... FOR UPDATE` before comparing, so concurrent writers serialise on that row. The migration runner already uses `sql.begin()` — the pattern exists in the codebase, it just is not used in the API. Stock is never decremented anywhere either, so this needs settling as part of designing checkout (M15).

HIGH

H4 — JWT configuration is never validated and verify accepts any algorithm the secret allows

Where `src/lib/auth/jwt.ts:9-23`, `src/lib/structures/Server.ts` (start)

Issue `JwtService` reads `JWT_SECRET` at construction with no check that it exists or is long enough, passes `Number(JWT_EXPIRES_IN)` to sign without validating it, and calls `jwt.verify` with no `algorithms` option. `start()` validates PORT and the database connection but neither JWT variable.

JWT.TS

```
private readonly secret = process.env.JWT_SECRET;
return jwt.sign(payload, this.secret, { expiresIn: Number(process.env.JWT_EXPIRES_IN) });
...
return jwt.verify(token, this.secret) as JWPayload;
```

Impact Three failure modes. If `JWT_SECRET` is missing the server still starts and passes its health check, then fails every login with a 500 and rejects every token with a 401 — a total outage that looks like an auth bug. If `JWT_EXPIRES_IN` is a duration string such as "1h" (the format the `jsonwebtoken` docs lead with, and what `typings/env.d.ts` declares as `string`), `Number()` yields `NaN` and `sign` throws, again only at request time. And a weak secret is accepted silently, which is what actually makes HS256 forgeable. Pinning algorithms closes the confusion attacks that arise if the key material ever changes.

Fix Parse the environment once at startup with a Zod schema — `JWT_SECRET` with a minimum length, `JWT_EXPIRES_IN` coerced and range-checked — and refuse to boot if it fails. Pass `algorithms: ['HS256']` to `verify`, and set `issuer` and `audience` so tokens cannot be replayed across environments.

HIGH

H5 — Registration is case-sensitive on email and races on duplicates

Where `src/api/auth/register.ts:8-13`, `userRepository.findByEmail`, `001_users.sql`

Issue Email is stored and compared verbatim. The duplicate check is a `SELECT` followed by an `INSERT`, with no unique-violation handling and no transaction.

REGISTER.TS

```
const existingUser = await userDb.findByEmail(registerBody.email);
if (existingUser) { res.writeHead(409); return res.end(...); }
const password_hash = await Password.hash(registerBody.password);
const newUser = await userDb.create({ ... });
```

Impact Two problems from one root cause. Because the UNIQUE constraint on `users.email` is case-sensitive, `Alice@example.com` and `alice@example.com` are two distinct accounts — a support and account-takeover hazard, and a login that silently fails for anyone who capitalised their address. And under concurrency both requests pass the existence check, the second INSERT raises 23505, and the client gets H1's leaked Postgres error with status 400 instead of the 409 the endpoint already knows how to return.

Fix Normalise on the way in with `z.email().toLowerCase().trim()`, add a migration for a unique index on `lower(email)` (or switch the column to `citext`), and drop the pre-check entirely — attempt the INSERT and translate 23505 to 409, exactly as `CategoryRepository.create` already does.

HIGH

H6 — `products.is_active` and `products.stock` are nullable, but every schema assumes they are not

Where `migrations/files/003_products.sql:6,11` and `src/lib/validation/productSchema.ts`

Issue Both columns are declared with DEFAULT but without NOT NULL. The Zod row schemas use `.default()`, which in Zod substitutes only for undefined — never for null.

003_PRODUCTS.SQL / PRODUCTSCHEMA.TS

```
is_active BOOLEAN DEFAULT TRUE,      -- nullable
stock      INT      DEFAULT 0,        -- nullable

is_active: z.boolean().default(true), -- null -> ZodError
stock:     z.int().default(0),         -- null -> ZodError
```

Impact Any row that gets an explicit NULL in either column — a bulk import, a manual fix-up, a future migration — becomes unreadable through the API: `dbProductSchema.parse` throws and the route answers 400. A NULL `is_active` is also excluded by every `WHERE is_active = TRUE` filter, so the product silently disappears from listings while still existing in the table. The CHECK (`stock >= 0`) constraint does not help, because NULL passes a CHECK.

Fix Add a migration setting both columns NOT NULL, backfilling any existing NULLs first. Keep the Zod `.default()` for insert paths but make the row schemas non-optional so drift like this fails loudly in tests rather than at a customer's request.

HIGH

H7 — No rate limiting or lockout on the authentication endpoints

Where `src/api/auth/login.ts`, `src/api/auth/register.ts`, `Server.ts`

Issue There is no per-IP or per-account throttle anywhere in the dispatch path, and no failed-attempt counter on the users table.

Impact POST `/api/auth/login` can be called as fast as the event loop allows. Combined with H2's absence of a password policy, online guessing against a known email address is entirely practical. `bcrypt` at cost 10 is roughly 100ms of CPU per attempt, which also makes the same endpoint an effective CPU-exhaustion target: a few hundred concurrent login attempts will saturate the single Node process and take the whole API down. Registration has the same exposure and can be used to flood the users table.

Fix Throttle before the `bcrypt` call, not after. A per-IP and per-email sliding-window counter in Postgres or Redis is enough; return 429 with `Retry-After`. Add progressive lockout on repeated failures against one account, and set `server.requestTimeout` and `server.headersTimeout` so slow clients cannot hold connections open.

Medium severity

MEDIUM**M1 — An empty PATCH body generates invalid SQL**

Where `productRepository.update`, `productSchema.ts` (`updateProductSchema`)

Issue `updateProductSchema` is `.partial()`, so `{}` is a valid payload. It reaches `SET ${this.db(data)}`, and the `postgres.js` update builder maps over `Object.keys` — for an empty object that produces an empty string.

CONFIRMED AGAINST THE INSTALLED DRIVER

```
// productSchema.ts
export const updateProductSchema = dbProductSchema
  .omit({ is_active: true, created_at: true, id: true }).partial();

// node_modules/postgres/src/types.js – the update builder
update(first, rest, ...) {
  return (rest.length ? rest.flat() : Object.keys(first)).map(x => ...)
}

// resulting statement
UPDATE products SET  WHERE id = $1 AND is_active = TRUE RETURNING ...
```

Impact `PATCH /api/products/1` with a body of `{}` reaches Postgres as syntax error 42601. Via H1 the client gets a 400 carrying the malformed statement — which is also the cleanest way for an attacker to confirm H1 exists. The same applies to a body whose every key is unknown, since Zod strips unknown keys before the builder sees them.

Fix Reject empty updates at the schema boundary: `.partial().refine(d => Object.keys(d).length > 0, 'At least one field must be provided')`. Zod's key-stripping is what keeps this from being an identifier-injection bug, so keep the schema between the request and the builder — never pass a raw body to `this.db()`.

MEDIUM**M2 — Deleting a cart item that is not in the cart returns 400 instead of 404**

Where `cartRepository.removeItem / updateQuantity`, `src/api/cart/[productId].ts:26-36`

Issue `removeItem` parses the returned row unconditionally. When the `DELETE` matches nothing, `item` is undefined and `dbCartItemSchema.parse(undefined)` throws a `ZodError`, which `Server.ts` turns into a 400 validation response.

CARTREPOSITORY.TS

```
const [item] = await this.db`DELETE FROM cart_items WHERE ... RETURNING *`;
return dbCartItemSchema.parse(item); // item === undefined -> ZodError
```

Impact The route checks that the product exists but never that the cart item does, so `DELETE /api/cart/:productId` for an item the user does not hold answers 400 with a Zod tree describing an internal row schema. A client cannot distinguish "your request was malformed" from "that item was not in your cart", and repeating a successful delete flips from 204 to 400 rather than being idempotent.

Fix Follow the pattern `findItem` already uses — `return item ? schema.parse(item) : null` — and have the route throw `CustomError(404)` on null. `ProductRepository.get` and `.update` already do exactly this; the cart repository is the inconsistent one.

MEDIUM**M3 — isNaN accepts non-integer ids, pushing bad values into integer columns**

Where `src/api/products/[id].ts:12,21,30` and `src/api/cart/[productId].ts:8,28`

Issue Every `id` is validated with `Number(...)` plus `isNaN`, which admits floats, exponent notation, negatives, Infinity and whitespace-padded values.

PRODUCTS/[ID].TS

```
const id = Number(params.id);
if (isNaN(id)) throw new CustomError(404, "product not found");
// Number("1.5") -> 1.5   Number("1e3") -> 1000
// Number("-4")  -> -4    Number(" 2 ") -> 2
```

Impact `GET /api/products/1.5` sends 1.5 into a comparison against an `INT` column and fails in the driver rather than returning 404, surfacing H1's leak. `/api/products/1e3` silently resolves to product 1000, so two distinct URLs address one resource — bad for caching and for audit trails. The auth middleware already gets this right with `Number.isSafeInteger(id) && id > 0`; the routes do not.

Fix Parse ids with a shared Zod schema — `z.coerce.number().int().positive()` — reused in both dynamic routes. That also removes the repeated `if (!params || !params.id) throw new Error(...)` guards, which currently produce a 500 for a condition the router makes impossible.

MEDIUM

M4 — price is validated only as a string, so any text reaches the NUMERIC column

Where `src/lib/validation/productSchema.ts:8` (`price: z.string()`)

Issue price is typed as a bare string in the row, insert and update schemas, with no numeric or decimal-format check.

Impact POST `/api/products` with `price: "abc"` or `"1e999"` passes validation and fails in Postgres, again via H1. `price: "-5"` is caught only by the CHECK constraint, and `"1.999"` is silently rounded by `NUMERIC(10,2)` rather than rejected. Modelling money as a string on the way out is defensible — it avoids float rounding, and `postgres.js` returns `NUMERIC` as a string — but it has to be validated as a decimal on the way in.

Fix Split the two directions: keep `z.string()` on the row schema that reads `NUMERIC` back, and validate input with a decimal pattern plus range, or `z.coerce.number().nonnegative().multipleOf(0.01)`. Document which representation the API returns, since `publicProductSchema` emits price as a JSON string while stock is a number.

MEDIUM

M5 — Shutdown closes the database pool before the HTTP server, failing in-flight requests

Where `src/lib/structures/Server.ts` (`shutdown`)

Issue The pool is ended first and awaited; only then is the HTTP server asked to close. There is no drain timeout and no handling of idle keep-alive sockets.

SERVER.TS

```
public async shutdown() {
  await DB.getInstance().close(); // pool ends first
  return new Promise((resolve, reject) => {
    super.close((err) => err ? reject(err) : resolve());
  });
}
```

Impact On SIGTERM — every rolling deploy, every container restart — requests already inside a handler lose their connection mid-query and return 500s, when the whole point of graceful shutdown is that they finish. Worse, `server.close()` only resolves once every connection is gone, so a single idle keep-alive client blocks the promise indefinitely and the orchestrator eventually SIGKILLS the process. `src/index.ts` awaits `shutdown()` before `process.exit(0)`, so it hangs there.

Fix Reverse the order and bound it: stop accepting connections, call `server.closeIdleConnections()`, wait for in-flight requests with a 10–15 second timeout, then close the pool, then exit. Guard against a second SIGTERM re-entering shutdown while the first is still running.

MEDIUM

M6 — 405 responses omit the Allow header, and OPTIONS and HEAD are unhandled

Where `src/lib/structures/Server.ts:29,36` and `resolveHandler`

Issue `req.method` is cast straight to the five-member `HTTPMethod` union. Unmatched methods get a bare 405 or 404 with no body and no Allow header.

SERVER.TS

```
const method = req.method as HTTPMethod; // OPTIONS and HEAD are not in the union
return res.writeHead(
  this.knownPaths.has(path) || this.hasDynamicMatch(path) ? 405 : 404
).end();
```

Impact RFC 9110 requires Allow on a 405; without it a client cannot discover what the resource supports. OPTIONS falls through to 404 or 405, so no browser preflight can ever succeed — any future browser client is blocked, and there is no CORS handling at all. HEAD is likewise unsupported, so health-checkers and proxies that probe with HEAD see 404. The cast is also simply unsound: the type claims a guarantee the runtime does not provide.

Fix Narrow the method with a runtime guard instead of a cast, and return 501 for methods you do not implement. Build the Allow set from the registry — the information is already in staticRoutes and dynamicRoutes — and answer OPTIONS from it. Add HEAD by dispatching to GET and suppressing the body.

MEDIUM

M7 — Multi-byte request bodies can be corrupted at chunk boundaries

Where `src/lib/structures/Parser.ts` (`parseBody`)

Issue Buffers are concatenated onto a string, so each chunk is decoded independently by the implicit `toString()` in `body += chunk`.

PARSER.TS

```
let body = "";
req.on("data", (chunk) => {
  bytes += chunk.length;
  ...
  body += chunk;      // implicit chunk.toString() per chunk
});
```

Impact A UTF-8 character split across a TCP boundary decodes as two replacement characters, one at the end of one chunk and one at the start of the next. The corrupted text either lands in the database or breaks `JSON.parse`, and the failure depends on packet timing — so it is intermittent and effectively impossible to reproduce from a test that emits a single chunk, which is what the existing parser tests do. Any non-ASCII product name or description is exposed to this.

Fix Collect chunks in an array and decode once with `Buffer.concat(chunks).toString('utf8')`, or pipe through a `StringDecoder`. Add a test that splits a multi-byte character across two emitted chunks to lock the behaviour in.

MEDIUM

M8 — Clients can set `is_active` when creating a product (mass assignment)

Where `src/lib/validation/productSchema.ts:12` (`newProductSchema`)

Issue `newProductSchema` omits only `id` and `created_at`, leaving `is_active` — the soft-delete flag — as a client-settable field.

PRODUCTSCHEMA.TS

```
export const newProductSchema = dbProductSchema.omit({ id: true, created_at: true });
```

Impact `POST /api/products/new` with `is_active: false` creates a product invisible to every read path, because all of them filter on `is_active = TRUE`. `ProductRepository.create` explicitly threads the value through (`data.is_active ?? true`), so it is wired end to end. The blast radius is limited today because the route is ADMIN-only, but the pattern is what matters: a lifecycle flag driven by request data. The equivalent hole on users is correctly closed — `registerSchema` cannot set role.

Fix Omit `is_active` from `newProductSchema` as `publicProductSchema` and `updateProductSchema` already do, and drop the parameter from `create`. Soft-delete state should change only through the `DELETE` route.

MEDIUM

M9 — The 98% coverage figure describes four files, not the codebase

Where `vitest.config.ts`, `tests/`

Issue The v8 provider only instruments modules the tests actually import. The summary reports 53 statements in total, against roughly 1,100 lines of source across 40 files.

VITEST RUN --COVERAGE

```
Statements : 98.11% ( 52/53 )    <- 53 statements, not ~1,100 lines
```

```
covered:  lib/Utils.ts, lib/auth/jwt.ts, middleware.ts, Parser.ts
invisible: all 10 API routes, all 4 repositories, Server.ts,
           registerRoute.ts, DB, Logger, CustomError,
           every validation schema
```

Impact The number reads as near-complete coverage of a tested project, which makes it a misleading signal on a dashboard or in a pull request. Real statement coverage over `src/` is somewhere between a few percent and ten percent. Every finding in this report sits in code no test touches — H1, H3, M1, M2 and M5 would each be caught by a single test against the layer they live in.

Fix Set `coverage.all: true` with `include: ['src/**/*.ts']` so uncovered files are counted, then add a coverage threshold floor and raise it as tests land. Prioritise integration tests over the dispatch path — a real `http.Server` on an ephemeral port plus a throwaway database or a stubbed `sql` tag — since that is where the density of untested behaviour is highest.

MEDIUM

M10 — npm test never exits, tests are excluded from typecheck, and there is no CI or linter

Where `package.json` (`scripts.test`), `tsconfig.json` (`exclude`), repository root

Issue `test` maps to `vitest` in watch mode. `tsconfig` excludes tests, so `npm run typecheck` never checks them. There is no workflow directory, no ESLint config, no README and no `.env.example`.

PACKAGE.JSON / TSCONFIG.JSON

```
"test": "vitest",                // watch mode - never exits
"exclude": ["tests", "dist"],    // tests are never typechecked

// strict-style flags present but commented out:
// "noUnusedLocals", "noImplicitReturns", "noFallthroughCasesInSwitch"
```

Impact `npm test` hangs forever in any non-interactive context, so this project cannot be gated in CI as written. Test files drift out of type safety — `tests/middleware.test.ts` already relies on `IncomingMessage` casts nothing verifies. With `noUnusedLocals` off, dead code accumulates unnoticed: `src/api/categories/new.ts` imports `ProductRepository` and never uses it. Without a README or `.env.example`, a new contributor has to read `Server.ts` and `typings/env.d.ts` to discover the five required environment variables.

Fix Add `"test": "vitest run"` and keep the watcher as `test:watch`. Add a second `tsconfig` that includes tests and run both in typecheck. Add ESLint with `@typescript-eslint`, turn the commented-out compiler flags back on, and add a CI workflow running typecheck, lint and test on every push. A README and `.env.example` are an hour of work with the highest onboarding return of anything in this report.

MEDIUM

M11 — Response shapes are inconsistent and error bodies are often empty

Where across `src/api/**`, `src/lib/structures/Server.ts`

Issue Collection endpoints return bare arrays while item endpoints return wrapper objects. 401, 403, 404 and 405 responses set `content-type: application/json` and then send no body at all.

OBSERVED RESPONSE SHAPES

```
GET /api/products/all -> [ {...}, {...} ] // bare array
GET /api/products/:id -> { product: {...} } // wrapped
POST /api/products/new -> { message, product } // wrapped + message
GET /api/cart -> [ {...} ] // bare array
401 / 403 / 404 / 405 -> content-type: json, zero-length body
```

Impact Clients need per-endpoint special cases, and a JSON `content-type` with an empty body makes well-behaved clients throw on parse. Returning a top-level array also leaves no room to add pagination metadata later without a breaking change. Error bodies vary between `{message}`, a Zod tree, a raw driver object and nothing at all, so no client can write one error handler.

Fix Pick one envelope and apply it everywhere, including the error paths in `Server.ts` — `{ data, meta }` for success and `{ error: { code, message, details? } }` for failure is a reasonable default. Send a JSON body with every error status, or do not set the `content-type` when there is no body.

MEDIUM

M12 — No pagination, filtering or sorting — and the query parameters are plumbed but never read

Where `src/api/products/all.ts`, `src/api/categories/all.ts`, repository `getAll()`

Issue Both list endpoints run an unbounded `SELECT`. `MethodOptions` carries a parsed query: `URLSearchParams` to every handler, and no handler in the codebase touches it.

```
PRODUCTREPOSITORY.GETALL
SELECT id, category_id, name, description, price, stock, created_at
FROM products
WHERE is_active = TRUE;    -- no LIMIT, no ORDER BY, no filter
```

Impact Response size and memory grow linearly with the table, so the endpoint degrades as the catalogue does — and a single request can pull the whole products table into the process. With no ORDER BY, row order is whatever Postgres chooses and can change between calls, making even client-side paging unreliable. For a product API, filtering by category and sorting by price are table stakes; there is currently no way to ask for either.

Fix Add limit and offset (or keyset pagination on created_at, id) with a server-side maximum, a deterministic ORDER BY, and optional category_id filtering — validated by a Zod schema over the query object already handed to every route.

4. Architecture

The shape of the project is its strongest quality. Routes are declarative Route instances that state their auth requirements per method; the dispatcher resolves, authorises, parses and then calls a handler that does nothing but validate, delegate to a repository and write a response. Repositories own all SQL and return parsed, typed rows. Zod schemas sit at both boundaries — request input and database output — which catches drift between the migrations and the code. For a from-scratch framework in about 1,100 lines, the seams are in sensible places, and the file-system route registry with its [param] convention is genuinely pleasant to use.

Four structural points are worth addressing before the codebase grows:

- Barrel-import cycle. Server.ts imports registerRoutes, DB and CustomError from #lib, and #lib re-exports Server.ts. ESM hoisting makes this work today, but the cycle is real and its failure mode — an undefined binding at module-evaluation time — is hard to diagnose. Import structures directly by path inside lib/, and reserve the #lib barrel for consumers in api/.
- Server extends http.Server and then hands itself to every handler as options.server. Any route can call this.close() or mutate the route tables, and no handler uses it. Drop the field, and prefer composition over extending the Node server — the class currently mixes transport, routing, error mapping and the database handle in one place.
- DB.initialize(server) takes the whole server purely to write one log line, coupling the data layer to the transport layer for no benefit. Pass a logger, or nothing. The singleton also makes it impossible to point two connections at different databases, which is the usual way integration tests get isolation.
- registerRoutes reads process.cwd() + '/dist/api' while importing via relative paths. This works only when the process starts from the project root against a build output, so the registry cannot be unit-tested and breaks under any other layout. Derive the directory from import.meta.url and use one path source for both the scan and the import.

Error mapping is centralised in the dispatcher, which is the right call — the problem in H1 is what the mapping does, not where it lives. Once driver errors are translated properly, that one function is the natural place to attach request ids and structured logging (M14).

5. REST and API design

The HTTP surface is the least finished part of the project. Paths encode actions rather than resources, the collection and item endpoints for one resource live at unrelated URLs, and there is no PUT anywhere despite the type union supporting it.

CURRENT	CONVENTIONAL	NOTE
GET /api/products/all	GET /api/products	The collection is the resource path; 'all' is redundant.

CURRENT	CONVENTIONAL	NOTE
POST /api/products/new	POST /api/products	The method already says 'create'.
GET /api/products/{id}	GET /api/products/{id}	Correct as-is.
GET /api/categories/all	GET /api/categories	Same as products.
POST /api/categories/new	POST /api/categories	No item route exists at all — no GET, PATCH or DELETE by id.
(none)	PUT /api/products/{id}	No full-replacement semantics anywhere in the API.
(none)	POST /api/orders	orders and order_items exist in the schema with no endpoints (M15).

Two further design points. GET /api/products and GET /api/categories both require a bearer token, so an anonymous visitor cannot browse the catalogue — unusual for a store, and worth confirming as deliberate rather than copied from the write routes. And the cart is addressed as /api/cart with the current user taken from the token, which is the right choice: it makes horizontal privilege escalation structurally impossible, since user.id never comes from the request path. That pattern is applied consistently across all four cart operations.

6. Data model and migrations

The schema is careful work. Every foreign key states its delete behaviour deliberately — RESTRICT on products.category_id and cart_items.product_id, CASCADE on cart_items.user_id — CHECK constraints guard prices and quantities, order_items records price_at_purchase so historical orders survive a price change, and cart_items has the UNIQUE (user_id, product_id) pair that makes the ON CONFLICT upsert correct. The migration runner is a genuinely good hundred lines: forward and rollback in one file, each applied inside sql.begin(), and a migrations table recording what ran.

Schema gaps

- Nullable columns the code treats as non-null — see H6. products.is_active and products.stock both need NOT NULL.
- No index on products.category_id. Postgres does not index the referencing side of a foreign key automatically, so every category filter and every RESTRICT check on category delete is a sequential scan.
- No partial index for the hot read path. Every product read filters WHERE is_active = TRUE; an index on that column, or a partial index over the active rows, will matter as the catalogue grows.
- No updated_at anywhere. products is mutable via PATCH with no record of when, which rules out change auditing and makes cache invalidation and ETags impossible.
- orders and order_items are dead schema — fully specified, with no code referencing either table (M15).
- cart_items.quantity is checked only for quantity > 0. Nothing at the database level ties it to product stock, so H3's oversell has no backstop.

Migration runner gaps

- down rolls back exactly one migration with no target, so undoing a multi-migration deploy means running it repeatedly and hoping the count is right.
- No advisory lock. Two instances starting concurrently — the normal case in a rolling deploy — can both read an empty applied set and both try to apply the same file. Wrapping the run in pg_advisory_lock fixes it.
- No checksums. The migrations table records only a name, so editing an already-applied file is undetectable. Store a hash of the contents and fail if it changes.
- parseMigration's /-- UP([lsS]*)-- DOWN([lsS]*)/ is greedy on the up section, so it splits on the last -- DOWN in the file. A migration whose SQL mentions -- DOWN in a comment will silently take the wrong split.

- The case blocks in script.js declare let count and const latest without braces, so all cases share one lexical scope. It happens to work; it is also exactly what the no-case-declarations rule exists to catch, and there is no linter to catch it (M10).

7. Security

The fundamentals are sound. Every query in every repository is a tagged template, so values are parameterised and the injection surface is closed — including the one dynamic construction, `SET ${this.db(data)}`, which is safe because Zod strips unknown keys before the builder could see them as identifiers. Passwords are bcrypt-hashed, never selected into a public response, and the login path verifies against a dummy hash when the email is unknown so response timing does not reveal whether an account exists. Authorisation is declared per method on each route and enforced centrally before any handler runs, and the middleware validates that the token subject is a positive safe integer rather than trusting the claim.

CONTROL	STATE	DETAIL
Parameterised SQL	Present	Tagged templates everywhere; no string concatenation into SQL.
Password hashing	Present	bcrypt, cost 10. Consider 12, re-hashing on next successful login.
Login timing defence	Present	Dummy-hash comparison on unknown email — a deliberate, correct touch.
Role enforcement	Present	Declared per method, checked centrally before the handler runs.
Horizontal isolation	Present	Cart identity comes from the token, never from the path.
Body size limit	Present	1 MB cap in Parser, enforced while streaming.
Password policy	Missing	H2 — empty passwords accepted.
Rate limiting / lockout	Missing	H7 — brute force and bcrypt CPU exhaustion both open.
Startup secret validation	Missing	H4 — a missing JWT_SECRET is discovered at request time.
Error-detail suppression	Missing	H1 — driver internals returned to unauthenticated callers.
Token revocation	Missing	No denylist or token version; a role change waits for expiry.
CORS / security headers	Missing	No CORS, HSTS, X-Content-Type-Options or frame options.
Request timeouts	Missing	Node defaults only; no requestTimeout or headersTimeout set.
Access logging	Missing	M14 — no per-request log, so no audit trail and no request ids.

One note on secrets handling: `.env` is correctly covered by `.gitignore` (`.env*`) and is not tracked in git — verified against git ls-files. There is no `.env.example` documenting the five required variables, which is the gap to close, not the ignore rule.

8. Lower-severity observations

These are worth fixing, but none changes the risk profile on its own.

ID	OBSERVATION	LOCATION
M13	Catalogue reads require a bearer token; likely unintended for a storefront. Confirm as deliberate.	products/all.ts categories/all.ts
M14	No access log or request ids — only route registration and errors are logged. LogLevel.Http is defined and never used, and the file transports have no rotation, so logs/ grows without bound.	Server.ts, Logger.ts

ID	OBSERVATION	LOCATION
M15	orders and order_items are fully migrated with no routes, repositories or checkout flow. Stock is never decremented anywhere.	migrations/files/005,006
M16	Missing index on products.category_id and no partial index for the is_active read path.	003_products.sql
L1	Non-RESTful paths and no PUT; categories has no item route at all. See section 5.	src/api/**
L2	Barrel-import cycle between Server.ts and #lib.	lib/index.ts, Server.ts
L3	Server extends http.Server and passes itself to every handler; no handler uses it.	Server.ts, Route.ts
L4	DB.initialize takes the Server only to write one log line.	lib/db/index.ts
L5	UserRepository.getAll() is unused and selects password_hash for every user. Delete it before someone wires it to a route.	userRepository.ts
L6	Unused import of ProductRepository; noUnusedLocals is commented out so nothing catches it.	api/categories/new.ts
L7	The route registry depends on process.cwd() and the dist/ layout, so it cannot be unit-tested.	registry/registerRoute.ts
L8	Migration runner: single-step down, no advisory lock, no checksums, greedy UP/DOWN split. See section 6.	migrations/script.js, utils.js
L9	bcrypt cost 10 (12 is the current recommendation); CustomError sets no name and exposes code only through a getter.	auth/hash.ts, CustomError.ts
L10	No README, no .env.example, no OpenAPI or API reference. A description already exists on every Route and could generate one.	repository root
L11	tsconfig sets jsx: react-jsx for a Node-only API; coverage is force-enabled on every vitest run, slowing the inner loop.	tsconfig.json vitest.config.ts
L12	204 responses carry content-type: application/json because the header is set unconditionally before dispatch.	Server.ts products/[id].ts
L13	Money crosses the API as a string for price but a number for stock, with no documented convention.	productSchema.ts
L14	Zod usage mixes flatten() (deprecated in v4) with treeifyError in the same error branch.	Server.ts
L15	tests/env.test.ts asserts against the developer's real .env, so the suite passes or fails on local machine state.	tests/env.test.ts

9. Recommended order of work

Sequenced so each step makes the next one safer to take. The first group is small and removes most of the production risk.

First — contain the risk (roughly a day)

- H1: stop serialising driver errors; map known codes and return a generic 500 for the rest. This single change also closes the information leak behind M1, M3, M4 and H5.
- H2: add a minimum password length to registerSchema, decoupled from loginSchema.

- H4: validate the environment with a Zod schema at startup and refuse to boot on failure; pin algorithms: ['HS256'] on verify.
- M5: reverse the shutdown order and bound the drain with a timeout.
- M10: switch test to vitest run and add a CI workflow — nothing below is safe to change repeatedly without it.

Next — correctness (two to three days)

- H3: wrap cart mutations in transactions with SELECT ... FOR UPDATE on the product row, and decide at the same time where stock is decremented.
- H5 and H6: migrations for a unique index on lower(email) and NOT NULL on products.is_active and products.stock; normalise email on input and drop the check-then-insert.
- M1, M2, M3, M4: reject empty PATCH bodies, return null from the cart repository and 404 from the route, and validate ids and price with shared Zod schemas.
- M7 and M8: buffer-then-decode in the parser; remove is_active from newProductSchema.
- H7: throttle the auth endpoints before the bcrypt call, and set request timeouts.

Then — make the gaps visible (ongoing)

- M9: coverage.all with an include glob and a threshold floor, then integration tests over the dispatch path — one per route, asserting status and body shape. Every High finding in this report would have been caught by one.
- M6, M11, M12: Allow and OPTIONS handling, one response envelope, pagination with a server-side maximum. These are breaking changes, so land them together and version the API if anything already consumes it.
- M16 and the section 6 migration hardening: indexes, advisory lock, checksums.
- L10: a README, a .env.example, and an OpenAPI document generated from the descriptions already declared on every Route.
- M15: finish orders, or delete the two migrations until the feature is real. Dead schema invites divergence.

10. What is working well

Worth stating plainly, because these are the parts to protect while changing the rest:

- The layering holds. Routes are thin, repositories own SQL, and validation sits at both boundaries. Almost every finding in this report is a small change inside one layer rather than a redesign across several — that is a direct consequence of the structure.
- The dummy-hash comparison on failed login is a deliberate defence against user enumeration that most projects at this scale get wrong.
- Validating database rows on the way out, not just requests on the way in, is unusual discipline and is exactly what will surface schema drift early.
- Deriving schemas from one row definition with omit and partial keeps the request contracts honest and made this review much faster. It is also why H6 and M8 are one-line fixes.
- The migration runner is small, transactional, has a real rollback path, and prints timings. Many teams reach for a heavyweight dependency and get less.
- The 35 existing tests are genuinely good — particularly the middleware suite, which covers wrong secrets, expired tokens, malformed subjects and case-insensitive schemes. The problem is coverage breadth, not test quality.
- Strict TypeScript is actually strict: noUncheckedIndexedAccess and exactOptionalPropertyTypes are both on, and the tree typechecks clean.